output, and the interval is updated using the same transformation as in the encoding process.

To ensure correct symbol retrieval in subsequent steps, X is normalized within the updated interval, and the process continues until the entire sequence is reconstructed.

4.4 Compression or Generation?

One might be curious about the necessity of using LLMs to compress LLM-generated instead of directly regenerating the text. In most cases, LLMs do not generate identical outputs for the same input due to multiple interacting factors that introduce variability. While LLMs are fundamentally deterministic in that they map a given input sequence to a set of probability distributions over tokens, the presence of stochastic sampling, architectural routing mechanisms, and computational nuances can lead to nondeterministic behavior. Even when all explicit sources of randomness, such as temperature and sampling, are eliminated, underlying numerical precision issues and architectural decisions may still cause slight variations in the generated text.

Temperature. One of the primary factors contributing to output variability is the temperature (τ), a parameter that modulates the probability distribution over possible next tokens. Mathematically, temperature scales the logits (z_i) corresponded to word (w_i), before applying the softmax function:

$$P(w_i) = \frac{e^{z_i/\tau}}{\sum_j e^{z_j/\tau}}. \tag{17}$$

When $\tau > 1$, the probability distribution becomes flatter, increasing the likelihood of selecting lower-probability tokens, which enhances diversity in the generated output. Conversely, as $\tau \to 0$, the softmax function increasingly favors the token with the highest probability, making the output more deterministic [22]. However, due to floating-point precision limitations, even a near-zero temperature may not always yield perfectly consistent results.

Sampling. In addition to temperature, sampling strategies also significantly influence output determinism. Techniques such as top-k sampling restrict token selection to the k most probable candidates [17], while top-p (nucleus) sampling dynamically adjusts the candidate set to include the smallest subset of tokens whose cumulative probability exceeds a threshold p [22]. Additionally, even deterministic decoding methods such as greedy decoding (where the model selects the most probable token at each step) or beam search[18] (which maintains multiple hypotheses before selecting the highest-probability sequence) can exhibit variations due to numerical rounding errors, computational parallelism, and subtle differences in implementation.

Architecture. The model's architecture can also cause the issue of non-determinism. In traditional dense Transformer models, such as GPT-3 [4] and LLaMA [48], every token undergoes a fixed computational pathway, ensuring a higher degree of determinism when sampling is disabled [48]. However, more recent models, such as GPT-4 [44], are believed to employ Mixture-of-Experts (MoE) architectures, where only a subset of specialized neural network components, or "experts" are activated for each token [27]. The routing mechanism that determines which experts are used can introduce variability for its probabilistic nature, which can be sensitive to minute numerical fluctuations [16].

5 Evaluation

In this section, we systematically evaluate the compression performance of various methods across multiple LLM-generated datasets. We begin by outlining the experimental setup. Next, we introduce the baseline methods, including traditional entropy-based, dictionary-based, and NN-based compression techniques, providing a comprehensive comparison against our proposed approach.

To better understand key factors influencing compression efficiency, we conduct in-depth analyses on chunk size, model size, and dataset scale, investigating their impact on compression ratios. Additionally, we compare LLMs with and without instruction tuning, exploring how post-training affects LLM-based compression. Finally, we categorize the data based on structural properties and statistical characteristics, offering insights into how different types of text influence compression efficiency.

5.1 Experimental Setups

In this section, we detail the datasets, preprocessing steps, and evaluation metrics used in our study.

5.1.1 Evaluation Datasets.

Wiki [41]. This dataset consists of human-written and LLM-generated Wikipedia introductions. The human-written samples are sourced from Wikipedia, while the machine-generated samples are produced using the GPT-3 model.

Article [41]. The Article dataset includes scientific research abstracts in both human-written and machine-generated formats. Human-written abstracts are sourced from existing scientific publications, while LLM-generated abstracts are produced using GPT-3.5.

Code [54]. This dataset was generated using Mixtral-8x7B. It contains programming problems and solutions across multiple languages, including Python, JavaScript, TypeScript, C++, and C.

Math [39]. The Math dataset contains approximately 200K grade school math word problems, designed to improve the mathematical reasoning capabilities of language models. The answers were generated using Azure GPT-4-Turbo, ensuring consistency across different types of problems.

Science [31]. The Science dataset consists of 20K problem-solution pairs across 25 physics topics, with 25 subtopics per topic and 32 problems per topic-subtopic pair. All problem-solution pairs were generated using GPT-4, making this dataset well-suited for evaluating the compression of structured scientific content.

Clinical [30]. This dataset is the official dataset for Asclepius (`arXiv`), formatted in a Clinical Note - Question - Answer structure. Synthetic clinical notes were generated from PMC-Patient case reports using GPT-3.5, followed by instruction-answer pairs for 157K synthetic discharge summaries.

Web. The Web dataset consists of movie critic reviews generated using ChatGPT [43], structured to mimic human-written reviews.

Table 4: Detailed Characteristics of Selected Large Language Models

Model	Size (B)	Type	Year	Organization	Context Length	Data Source	Use Case
OpenELM-1_1B [36]	1.1B	Decoder	2024	Apple	4096	Web, Books	Efficient NLP tasks
AMD-OLMo-1B [33]	1B	Decoder	2024	AMD	4096	Code, Web Data	Optimized AI applications
Llama-3.2-1B [38]	1B	Decoder	2024	Meta AI	8192	Web, Academic Papers	General NLP
Llama-3.2-1B-Instruct [38]	1B	Decoder	2024	Meta AI	8192	Web, Academic Papers	Instruction-tuned NLP
Llama-3.2-3B [38]	3B	Decoder	2024	Meta AI	8192	Web, Books, Papers	Open-source NLP
Llama-3.2-3B-Instruct [38]	3B	Decoder	2024	Meta AI	8192	Web, Books, Papers	Instruction-tuned NLP
Llama-3.1-8B [37]	8B	Decoder	2024	Meta AI	16384	Large-scale Corpora	Advanced reasoning
Llama-3.1-8B-Instruct [37]	8B	Decoder	2024	Meta AI	16384	Large-scale Corpora	Instruction-following NLP
Qwen2.5-14B [46]	14B	Decoder	2024	Alibaba	32768	Multilingual Corpora	General NLP
Qwen2.5-14B-Instruct-1M [46]	14B	Decoder	2024	Alibaba	32768	Multilingual Corpora	Instruction-tuned NLP
Qwen2.5-Math-1.5B [55]	1.5B	Decoder	2024	Alibaba	16384	Mathematical Data	Mathematical reasoning
Rho-Math-1B [32]	1B	Decoder	2024	Microsoft	2048	Mathematical Data	Mathematical problem
Qwen2.5-Coder-1.5B [25]	1.5B	Decoder	2024	Alibaba	16384	Code Corpora	Code generation
DeepDeek-Coder-1.3b-Instruct [21]	1.3B	Decoder	2024	DeepSeek AI	16000	Code, Natural Language	Code completion

Novel. The Novel dataset consists of a travel book generated using LongWriter [2], simulating the style and structure of human-authored long-form narratives.

5.1.2 Evaluation Metrics. For lossless compression, the compression ratio is the primary performance metric. It quantifies the reduction in data size achieved by compression and is defined as the ratio between the original data size and the compressed data size:

$$R = \frac{S_{\text{original}}}{S_{\text{compressed}}} \tag{18}$$

where S_{original} is the size of the uncompressed data, and $S_{\text{compressed}}$ is the size of the data after applying a compression algorithm. A higher compression ratio indicates greater space savings, making storage and transmission more efficient.

5.1.3 Hardware Platform. Our experiment is conducted on a cluster powered by an AMD EPYC 7742 64-Core Processor equipped with a NVIDIA A100 Tensor Core GPU.

5.2 Baselines

To thoroughly investigate the compression characteristics of LLM-generated data, we introduce nine baselines spanning three categories: entropy-based, dictionary-based, and neural-based methods. Additionally, to examine the impact of different LLMs on the proposed framework, we include 14 LLMs in this comprehensive study.

5.2.1 Entropy-based Compressor. Huffman coding, Arithmetic coding and Finite State Entropy (FSE) is selected as entropy-bassed baselines. Huffman coding [24] is a foundational entropy coding technique, assigns shorter codes to more frequent symbols and serves as the basis for many compression algorithms. Finite State Entropy (FSE) [9] is an entropy coding method that dynamically models symbol probabilities using a finite-state machine, improving efficiency over fixed-length coding schemes like Huffman coding. Arithmetic coding [51] represents an entire message as a fractional number in the interval $[0, 1)$, refining the interval based on symbol probabilities to achieve near-optimal compression.

5.2.2 Dictionary-based Compressor. LZMA (Lempel-Ziv-Markov chain algorithm) [58] extends LZ77 with probability modeling and range coding, achieving high compression ratios at the cost of increased computational complexity. Gzip [15], based on the DEFLATE algorithm, integrates LZ77 for dictionary compression with Huffman coding for entropy encoding, offering a well-balanced tradeoff between compression ratio and speed, widely used for file compression and web data transfer. Zstandard (Zstd) [10], developed by Facebook, is a high-performance compression algorithm that combines LZ77-style compression with Finite State Entropy (FSE) coding, balancing speed and compression ratio while supporting dictionary-based compression.

5.2.3 NN-based Compressor. We adopt NNCP [3], TRACE [34] and PAC [35] as our NN baselines. Neural Network-based Compression (NNCP) utilizes deep learning models, such as autoencoders and transformers, to learn compact representations of data. TRACE designed a slim transformer to accelerate the compression procedure. PAC further builds an order model to replace attention to achieve faster compression speed.

5.2.4 LLM-based Compressor. In this paper, we incorporate LLMs of different sizes and from various companies, covering a range from 1B to 14B parameters. In deep learning, 250M, 1B, and 11B refer to models with 250 million, 1 billion, and 11 billion parameters, respectively. Each parameter is typically stored as a 16-bit (FP16) or 32-bit (FP32) floating-point number.

The selected models provide a diverse comparison across different model sizes and organizations. Specifically, we included multiple 1B-sized models, OpenELM-1.1B, AMD-OLMo-1B, and Llama-3.2-1B, to evaluate performance under similar computational constraints. The Llama series was chosen to investigate scaling laws in compression tasks, given its variations in size and training methodology. Furthermore, models from various companies, including Apple, AMD, Meta AI, and Alibaba, were selected to assess the generalization of our proposed method across different architectures and datasets.

Additionally, we include instruction-tuned versions of several models, which are aligned to human preferences, including Llama-3.2-1B-Instruct, Llama-3.2-3B-Instruct, Llama-3.1-8B-Instruct, and Qwen2.5-14B-Instruct-1M. Compared to their base counterparts, these models demonstrate enhanced usability in real-world interactive AI applications. In the following experiments, unless otherwise specified, we use Llama-3.1-8B as the base LLM.

Table 5: Compression results for various LLM-generated synthetic datasets.

	Entropy-based Compressor							
Method	Wiki	Code	Math	Clinical	Web	Science	Novel	Artical
Huffman	1.69	1.62	1.65	1.82	1.77	1.64	1.76	1.81
Arithmetic	1.70	1.63	1.66	1.61	1.77	1.64	1.77	1.81
FSE	1.68	1.62	1.65	1.83	1.77	1.64	1.75	1.80
	Dictionary-based Compressor							
Method	Wiki	Code	Math	Clinical	Web	Science	Novel	Artical
Gzip	2.73	3.47	4.30	5.70	4.26	5.20	3.40	3.48
Lzma	4.24	4.90	5.99	9.76	6.49	7.13	5.74	5.34
Zstd-22	4.17	4.85	5.90	9.95	6.46	7.05	5.68	5.31
	Neural-based Compressor							
Method	Wiki	Code	Math	Clinical	Web	Science	Novel	Artical
NNCP	5.04	6.30	7.76	12.56	2.95	7.38	5.39	6.30
TRACE	4.11	4.53	5.34	9.55	4.75	5.09	5.09	5.53
PAC	4.17	5.08	6.39	9.94	3.48	6.66	5.02	5.63
Ours	**14.62**	**20.44**	**15.22**	**17.84**	**22.26**	**23.80**	**23.13**	**16.97**

5.3 Overview of Compression Performance Across LLM-Generated Datasets

In this subsection, we provide a comprehensive analysis of compression efficiency across various LLM-generated synthetic datasets. Table 5 presents the results obtained using different categories of compression methods, including entropy-based, dictionary-based, and neural-based approaches. The evaluation covers a diverse range of datasets, including Wiki, Code, Math, Clinical, Web, Science, Novel, and Article, allowing for a thorough comparison of how different methods handle distinct types of LLM-generated text.

The entropy-based compressors, including Huffman coding, Arithmetic coding, and FSE coding, exhibit the lowest compression performance across all datasets. Their limited ability to exploit long-range dependencies in text restricts their effectiveness, resulting in compression ratios consistently below 2.0. This highlights the fundamental limitation of statistical coding techniques when applied to highly structured, synthetic text.

Dictionary-based methods, such as Gzip, LZMA, and Zstd-22, show a significant improvement over entropy-based approaches, with compression ratios ranging from approximately 2.7 to 9.9 across different datasets. Among them, LZMA consistently achieves the highest compression rates, particularly on structured datasets such as Clinical and Science, where it benefits from repetitive patterns. However, despite their improved efficiency, these methods still fall short of neural compression approaches in leveraging deeper contextual information.

Neural-based compressors, including NNCP, TRACE, and PAC, further enhance compression performance, demonstrating that learned models can better capture the statistical structure of LLM-generated text. NNCP achieves particularly high compression rates on complex datasets such as Math and Science, while PAC and TRACE show competitive results across most categories. Notably, neural approaches tend to perform better on more structured datasets, suggesting that their learned representations effectively exploit latent regularities in LLM-generated content.

The most striking result comes from the proposed method, which significantly outperforms all baseline compressors across every dataset. The compression rates achieved by our approach are consistently higher than those of both traditional and neural-based compressors, demonstrating its superior ability to model LLM-generated data. Notably, the largest improvements are observed on Science, Clinical, and Novel datasets, where compression rates surpass 20. This indicates that our method effectively captures and exploits the structured patterns inherent in LLM-generated text, leading to unprecedented gains in compression performance.

5.4 Impact of Chunk Size on LLM Compression Peformance

Chunk size significantly influences LLM compression performance by shaping how contextual information is utilized for token prediction. This subsection examines the effect of varying chunk sizes across different model configurations, including both base and instruction-tuned variants. Specifically, we test Llama-3.2-1B, Llama-3.2-3B, and Llama-3.1-8B, along with their instruction-tuned counterparts, across chunk sizes ranging from 16 to 256.

Experimental results for Llama-3.2-1B, Llama-3.2-3B, and Llama-3.1-8B, along with their instruction-tuned counterparts, consistently show that larger chunk sizes yield higher compression rates. This aligns with the theoretical premise that a broader context enables more accurate token predictions, reducing redundancy in the encoded representation. Compression efficiency improves progressively from chunk size 16 to 256, with larger models such as Llama-3.1-8B exhibiting steeper gains. Instruction-tuned models systematically outperform their base counterparts, particularly at larger chunk sizes, indicating that instruction tuning enhances token predictability. However, compression gains diminish beyond a certain threshold. The most notable improvements occur between chunk sizes 16 and 64, while gains taper off beyond 128, suggesting diminishing returns from additional context.

Although increasing the size of the fragment generally improves compression, the gain is not strictly linear. The most significant improvements are observed when transitioning from chunk size 16 to 64, whereas the relative improvement diminishes as chunk size increases beyond 128. This indicates that beyond a certain